

CS452: Intro to DL Internals

Assignment 3: Building and Training a Neural Network

Instructor: Ram Vikas, PhD

Objective

This is it! We will take our powerful `Scalar` object and use it to build the components of a neural network: `Neurons`, `Layers`, and a full MLP (Multi-Layer Perceptron). We will then define a simple dataset, calculate a loss, and write a training loop that uses our from-scratch backpropagation to "learn".

Part A: Building the Network Components (Approx. 60 minutes)

A neural network is a collection of neurons organized in layers. Each neuron performs a simple calculation: $y = f(\sum w_i x_i + b)$, where the weights (w_i) and bias (b) are the learnable "knobs" of the network.

Task 1: The 'Neuron' Class Your task is to build a class representing a single neuron.

- Define a class named `Neuron`.
- The `__init__` method should accept `num_inputs`, the number of inputs the neuron will receive.
- Inside `__init__`, you must initialize the neuron's parameters. These are its "knobs".
 - Create an attribute `self.weights`, which should be a list of `Scalar` objects, one for each input. Initialize their data with random values between -1 and 1.
 - Create an attribute `self.bias`, which should be a single `Scalar` object, also initialized with a random value.
- Implement the `__call__(self, x)` method, which performs the neuron's forward pass. It should compute $\sum(w_i x_i) + b$. This entire calculation must use your `Scalar` operations. The final result should be passed through an activation function like `tanh`.
- Side-Quest:** You will need to add a `tanh` method to your `Scalar` class! Remember to also implement its local `_backward` function. The derivative of $\tanh(x)$ is $1 - \tanh(x)^2$.
- Add a helper method `parameters(self)` to the `Neuron` class that returns a single list containing all of its weight `Scalars` and its bias `Scalar`.

Task 2: The 'Layer' and 'MLP' Classes Now, let's organize our neurons into layers and stack the layers into a network.

- Define a `Layer` class. Its `__init__` should accept `num_inputs` and `num_neurons` and create a list of `Neurons`. Its `__call__` should apply all its neurons to an input `x`. Its `parameters()` method should collect and return the parameters from all its neurons.
- Define an `MLP` class (Multi-Layer Perceptron). Its `__init__` should accept the number of network inputs and a list of layer sizes (e.g., `[4, 4, 1]` for two hidden layers of 4 neurons and one output neuron). It should create a list of `Layers`.

- c. The MLP's `__call__` method should perform a full forward pass, passing the input sequentially through each layer.
- d. The MLP's `parameters()` method should return a flat list of all parameters from all layers in the network.

Part B: The Training Loop (Approx. 60 minutes)

This is the core "learning" process. It's a loop that repeats four steps: Forward Pass, Calculate Loss, Backward Pass, and Update.

Task 3: Implementing the Training Loop Write a script that trains your newly created MLP.

- a. **Setup:**
 - Create a small dataset: a list of input lists (`xs`) and a list of corresponding target floating-point values (`ys`).
 - Instantiate your MLP.
 - Define hyperparameters: a `learning_rate` (e.g., 0.05) and the number of `epochs` (e.g., 20).
- b. **The Loop:** Create a `for` loop that iterates for the number of epochs. Inside this loop, perform the following four steps in order:
- c. **Step 1: Forward Pass.** Get the model's predictions (`y_pred`) for every input in your dataset `xs`.
- d. **Step 2: Calculate Loss.** Compute a single `Scalar` value representing how "wrong" the predictions are. A good choice is Mean Squared Error: the average of $(y_{true} - y_{predicted})^2$ over all data points.
 - **Side-Quest:** You may need to add more special methods to your `Scalar` class, like `__sub__` and `__pow__`, to make the loss calculation clean.
- e. **Step 3: Backward Pass.**
 - **Crucially**, you must first zero out the gradients of all parameters in your model. Create a loop through `mlp.parameters()` and set `p.grad = 0.0` for each parameter `p`.
 - Call the `backward()` method on your final loss `Scalar`.
- f. **Step 4: Update Parameters.** Loop through all parameters in `mlp.parameters()` again. For each parameter `p`, update its data using the rule of gradient descent: `p.data -= learning_rate * p.grad`.
- g. **Logging:** Inside the loop, print the epoch number and the loss value (`loss.data`) to monitor the training progress.

Conclusion

Absolutely incredible! If your loss value decreases over the epochs, you have successfully built a complete neural network from scratch. You implemented the forward pass, the backward pass, and a training loop to optimize the network's parameters. You now understand the fundamental mechanics that power even the most complex AI models. Well done!